

# Mid-Year Technical Report

## 1. Potential (Unimplemented) TSFresh Improvements

One of the primary challenges associated with integrating TSFresh into a machine learning pipeline is its computational runtime. TSFresh is designed to generate a comprehensive set of time-series features during the feature extraction stage, often producing a very high-dimensional feature space. While this extensive feature generation can improve model performance, it also introduces significant computational overhead. Furthermore, TSFresh's feature selection process prioritizes statistical robustness and predictive relevance over computational efficiency, selecting the most appropriate statistical tests for the characteristics of the data rather than the fastest to execute. As a result, both the feature extraction and feature selection stages can become major contributors to the overall runtime of the pipeline.

Another improvement known involves the Pandas dataframes TSFresh uses, with possible changes to Polars or NumPy known to improve runtime from preliminary tests. Exact experiments have not been performed yet, and will not be discussed in this report.

### 1.1 Fourier Coefficients

Fourier coefficient features contribute substantially to TSFresh's computational cost. By default, TSFresh extracts 400 Fourier coefficient features, comprising 100 coefficients for each of the real, imaginary, magnitude, and phase representations of the discrete Fourier transform.

Increasing the number of coefficients provides a more detailed frequency-domain representation of a time series, but this additional information does not necessarily improve predictive performance. In many applications, lower-frequency coefficients capture most of the useful signal, while higher-frequency coefficients primarily describe fine detail or noise. Consequently, reducing the number of extracted coefficients may significantly decrease extraction time while having little impact on model accuracy.

As the real, imaginary, magnitude, and phase features represent different forms of the same Fourier coefficients, removing an entire category would discard one aspect of every frequency component. Instead, this investigation reduces the number of coefficients extracted from all four categories simultaneously, preserving a complete representation of the retained frequencies while lowering computational cost. In the following section, a model stated as “25FFT” would contain 25 frequency components within each of the four categories.

TSFresh provides three predefined extraction configurations. Minimal extracts a small subset of inexpensive features and excludes Fourier coefficients entirely. Efficient balances computational cost and predictive performance while including Fourier coefficient features, whereas Comprehensive extracts all available features.

The following experiments evaluate the impact of reducing the number of Fourier coefficients within these configurations, with particular focus on feature extraction time and classification accuracy.

### 1.1.1 Fourier Coefficients – Results

Testing was performed across 20 different datasets, using the same feature selector from TSFresh to generate what features are best used for prediction. Statistical tests were performed to justify whether confidence of differences can be seen. Important points will be stated below, in reference to the statistical results generated.

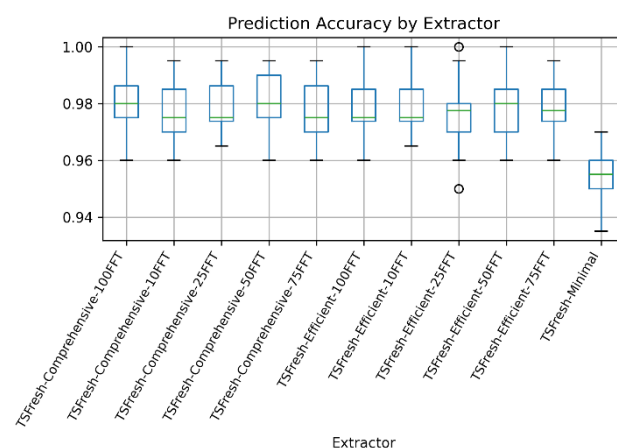


Figure 1: Prediction Accuracy on Dataset given TSFresh Extractor Type

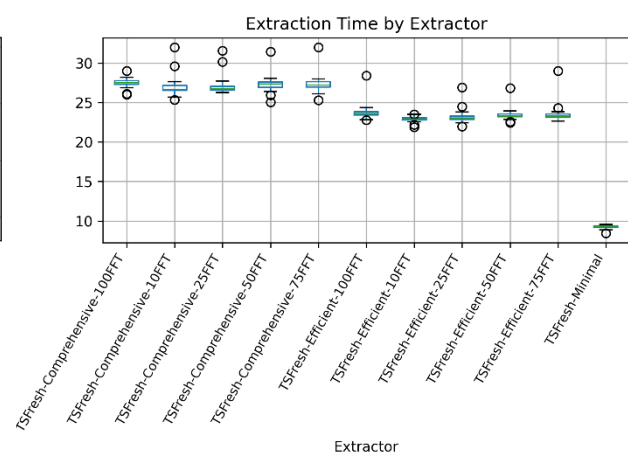


Figure 2: Feature Extraction Time on Dataset given TSFresh Extractor Type

From Figure 1, there is statistical evidence that the Minimal TSFresh configuration produces lower classification accuracy than both the Efficient and Comprehensive configurations, with an average reduction of approximately 3%. However, there is no statistical evidence to suggest that increasing the number of Fourier coefficients within the Efficient or Comprehensive configurations results in a significant improvement in classification accuracy. At the time of writing, the smallest number of Fourier coefficients evaluated was 10, with experiments conducted using 10, 25, 50, 75, and 100 coefficients. Consequently, it remains possible that performance differences exist at lower coefficient counts or between the tested values that were not captured by these experiments, noting that these experiments are still in a preliminary phase.

Figure 2 demonstrates that the Minimal configuration is statistically significantly faster than both the Efficient and Comprehensive configurations, reducing extraction time by approximately 14 and 16 seconds on average, respectively. The results also indicate that the Comprehensive configuration is significantly slower than the Efficient

configuration by an average of approximately 2 seconds. Within each configuration, there is no statistical evidence that varying the number of Fourier coefficients has a significant impact on extraction time.

It is important to note that these experiments were conducted using multiple random seeds applied to a consistent synthetic data generation process with a fixed dataset size and structure. While this provides a controlled basis for comparison, the measured extraction times should not be interpreted as universally applicable, as feature extraction time depends on dataset characteristics such as the number of instances and time-series length. For this reason, relative performance provides a more meaningful comparison than absolute execution time. On average, the Efficient configuration required approximately 2.4 times the extraction time of the Minimal configuration, while the Comprehensive configuration required approximately 2.6 times the extraction time of the Minimal configuration. Similarly, the Comprehensive configuration was approximately 1.08 times slower than the Efficient configuration. These ratios are representative only for the datasets used in this study. As dataset size increases, the computational complexity of the different configurations may scale differently, meaning these relative performance differences may change rather than remain constant.

## 2. Alternative Library Tests

A wide range of Python libraries are available for feature extraction and feature selection in time-series analysis. While many libraries specialise in one of these tasks, relatively few provide both capabilities. TSFresh is one of the most widely adopted libraries because it offers comprehensive implementations of both feature extraction and feature selection within a single framework.

Using the benchmarking methodology described in Section 1.1, a second set of experiments was conducted to compare a selection of feature extraction and feature selection methods. Based on the results presented in Section 1.1, only the Minimal and Efficient (50 Fourier coefficients) TSFresh extraction settings were included in these experiments, as it shows a reasonable approximation of TSFresh coverage given this dataset, and running time would increase with more libraries added.

The feature extraction methods evaluated were:

- TSFEL (Time Series Feature Extraction Library) – A dedicated library for extracting statistical, temporal, and spectral features from time-series data.
- Statistics – A custom extractor developed for this project that calculates five simple statistical features: mean, standard deviation, minimum, maximum, and energy.

- TSFresh Minimal – The Minimal TSFresh parameter set.
- TSFresh Efficient (50 Fourier coefficients) – The Efficient TSFresh parameter set configured to extract 50 Fourier coefficients.

The feature selection methods evaluated were:

- SelectKBest – A scikit-learn feature selector configured to retain the 20 highest-scoring features.
- Boruta – A wrapper-based feature selection algorithm that uses Random Forest feature importance to identify relevant features.
- TSFresh – TSFresh's built-in feature selection procedure, which applies statistical hypothesis tests to determine feature relevance.
- None – No feature selection is applied, allowing all extracted features to be used during classification.

The benchmarking framework was designed to allow any feature extraction method to be paired with any feature selection method, enabling direct comparison of all possible combinations. Consequently, four feature extraction methods and four feature selection methods were evaluated. The key findings and their implications are discussed below with respect to the statistical outputs.

## 2.1 Alternative Library Tests – Results

From Figure 3, there is statistical evidence that both the TSFEL and Statistics extractors have significantly lower extraction times than either TSFresh parameter setting. On average, TSFEL required approximately half the extraction time of the TSFresh Minimal extractor, while the Statistics extractor was approximately 1,000 times faster. This result is expected, as the Statistics extractor consists of a small number of directly calculated Python statistics, resulting in an average extraction time of only 0.0097 seconds for the benchmark dataset.

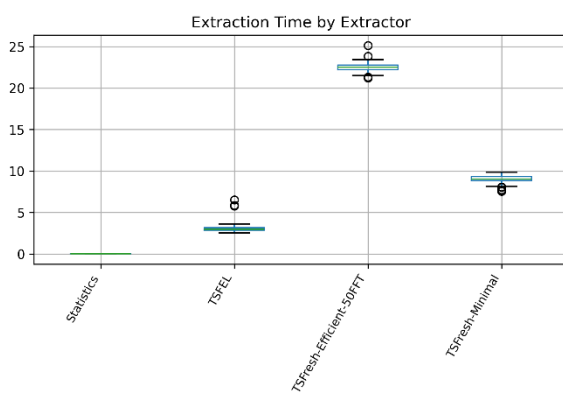


Figure 3: Extraction Time on Dataset given by Feature Extraction Method

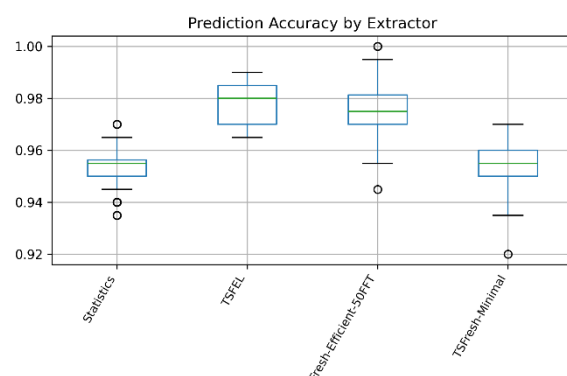


Figure 4: Prediction Accuracy on Dataset given by Feature Extraction Method

Figure 4 shows that both the TSFEL and TSFresh extractors achieve higher prediction accuracy than the Statistics and TSFresh Minimal extractors. Although TSFEL achieved a statistically significant improvement over TSFresh, the observed difference was approximately 0.2%, indicating that the practical benefit is negligible for this dataset. When considered alongside the substantially lower extraction time, TSFEL provides a more favourable balance between computational cost and predictive performance. In contrast, both the Statistics and TSFresh Minimal extractors produced prediction accuracies approximately 4% lower than TSFEL and TSFresh, with no statistically significant difference observed between these two methods. This is consistent with the TSFresh Minimal parameter set containing many fundamental statistical features similar to those calculated directly by the Statistics extractor.

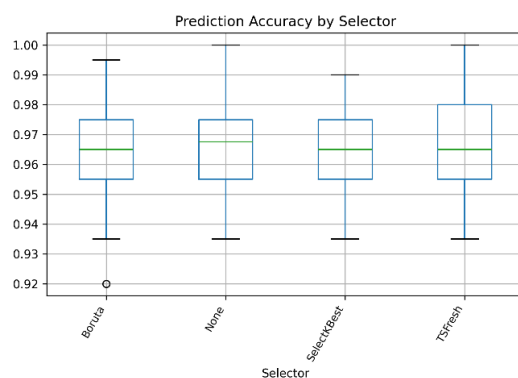


Figure 5: Prediction Accuracy on Dataset given by Feature Selection Method

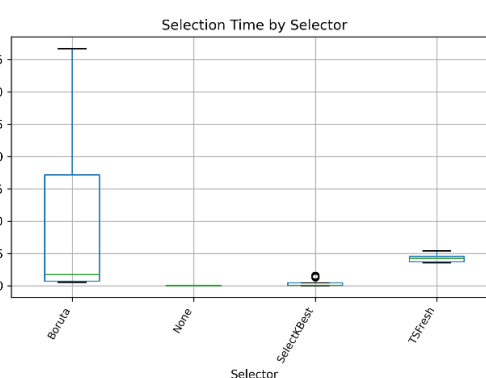


Figure 6: Selection Time on Dataset given by Feature Selection Method

Figure 5 indicates that there is no statistical evidence of a difference in prediction accuracy between any of the evaluated feature selection methods. Therefore, none of the selectors provided a measurable improvement in classification performance for the benchmark dataset. In contrast, Figure 6 demonstrates statistically significant differences in feature selection time between all methods. As expected, applying no feature selection incurred no computational cost. On average, SelectKBest required approximately 0.4 seconds, TSFresh feature selection required approximately 4 seconds, and Boruta required approximately 10 seconds.

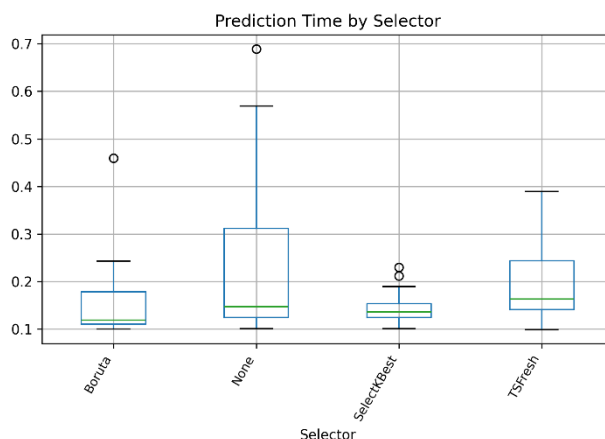


Figure 7: Prediction Time on Dataset given by Feature Selection Method

Figure 7 shows statistically significant differences in prediction time between the feature selection methods. Prediction was slowest when no feature selection was applied, as the classifier operated on the full feature set. Prediction time decreased as more aggressive feature selection reduced the number of input features, with SelectKBest producing the shortest prediction times, followed by Boruta, TSFresh, and finally no feature selection. However, the absolute differences between these methods were all less than 0.2 seconds, suggesting that, for datasets of this size, the practical impact of prediction time is minimal compared with the differences observed in feature extraction and selection time.

Figures 8 and 9 demonstrate that the performance of each extractor-selector combination is largely determined by the characteristics of the chosen extractor rather than any interaction between the two methods. The observed differences in prediction accuracy closely mirror those discussed for the extractors alone, while total execution time is consistent with the combined costs of feature extraction, feature selection, and prediction. No evidence was found to suggest that any particular extractor and selector combination exhibited unexpected incompatibilities or performance penalties beyond the sum of their individual computational costs.

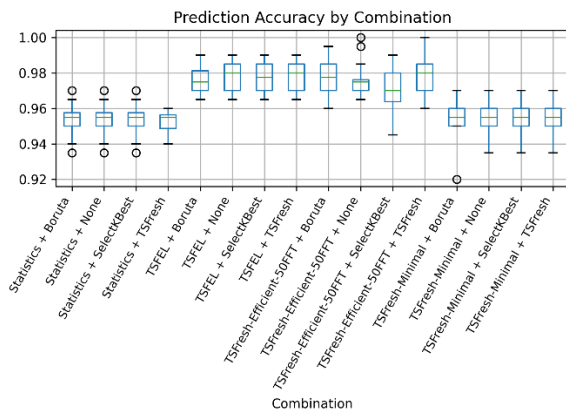


Figure 8: Prediction Accuracy on Dataset given by Combination of Feature Extraction and Selection methods

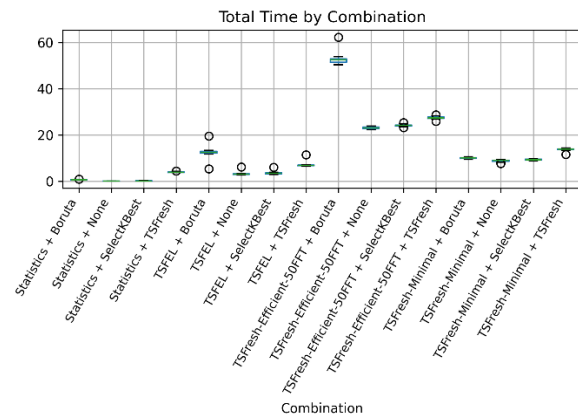


Figure 9: Total Runtime given by Combination of Feature Extraction and Selection methods

Overall, for the benchmark dataset used in this study, the TSFEL extractor without feature selection provided the most favourable trade-off between computational efficiency and predictive performance. It achieved the highest prediction accuracy while requiring substantially less extraction time than the TSFresh extractors. Although omitting feature selection resulted in a slightly longer prediction time due to the larger number of features presented to the classifier, this increase was small relative to the time saved during feature extraction. These findings are specific to the synthetic dataset and benchmark configuration used in this study. Future work should investigate additional feature extraction libraries, explore optimisations to TSFresh as discussed in Section 1, and evaluate the methods on larger and more diverse datasets to determine whether these conclusions generalise to other time-series classification problems.